

ML System Design Round — Ads Matching/Sourcing

Framework: How to Structure Your 45 Minutes

Per recruiter: "You'll be posed a problem that requires an ML system to solve it. You will run through and design the system, focusing on the modeling aspect but also the broader system as it relates to platform and infrastructure."

Their expectation: Modeling-heavy, but must show end-to-end system awareness. Not purely theoretical — they want to see you can BUILD this.

Time allocation:

Phase	Time	What to cover	Interviewer interaction
Clarification	3-5 min	Scope, scale, constraints	Ask questions
Metrics	3 min	Offline + online metrics	Confirm alignment
Architecture	5 min	Draw the pipeline	Get buy-in before diving
Data & Features	5 min	Training data, features	They may redirect
Modeling	15 min	Model architecture, training, loss	This is where they drill
Serving	7 min	Latency, scaling, embedding index	Practical details
Evaluation	5 min	Offline eval, A/B test, iteration	Wrap up

Key principle: At each phase, state what you'll cover, ask if they want you to go deeper or move on. This gives them control and shows you can lead a technical discussion.

Step 1: Problem Clarification (3-5 min)

How to say it:

"Before I dive in, let me clarify a few things about the problem scope."

Questions to ask (pick 4-5 relevant ones):

Question	Why it matters
"What's the scale? How many apps/ads and how many queries per second?"	Determines whether exact search is feasible or need ANN
"What's the latency budget?"	Constrains model complexity at serving time
"Is this a new system from scratch or improving an existing one?"	Affects architecture choices
"What signals do we have? Search query, user context, session history?"	Drives feature engineering
"What's the primary business objective? Revenue, relevance, user retention?"	Shapes the optimization objective

"Are there privacy constraints on what user data we can use?"	Critical for Apple
"Do we have labeled data? Human annotations? Click logs?"	Determines training approach
"How often do new apps/ads enter the system?"	Cold-start considerations

How to say it:

"Great, so let me summarize: we're designing an ad matching system for the App Store with [X] apps, [Y] QPS, latency under [Z]ms, privacy-constrained with no cross-app tracking, and the goal is to maximize relevance while driving ad revenue. Let me start by defining how we'd measure success."

Step 2: Metrics Definition (3 min)

How to say it:

"Let me define success metrics at two levels — offline for model development iteration, and online for business impact validation."

Offline metrics (model development):

Metric	Stage	What it measures
Recall@K	Retrieval	Coverage — did we find the relevant ads?
Precision@K	Retrieval	Quality — are retrieved ads relevant?
NDCG@K	Ranking	Position-aware ranking quality
PR-AUC	Relevance model	Relevance classification quality (NOT ROC-AUC — imbalanced data)
MRR	End-to-end	How quickly the first relevant ad appears

Online metrics (A/B test):

Metric	Stakeholder	What it measures
CTR	Platform + Advertiser	User engagement with ads
CVR (install rate)	Advertiser	Conversion quality
Revenue per query	Platform	Monetization
ROAS	Advertiser	Return on ad spend
Relevance (human eval)	User experience	Ad quality perception
Advertiser retention	Long-term health	Ecosystem sustainability

How to say it:

"For offline evaluation, I'd use Recall@K for retrieval coverage and NDCG for ranking quality. Importantly, I'd use PR-AUC over ROC-AUC because our problem is highly imbalanced — out of 50K apps, only a few hundred are relevant to any given query, so true negatives dominate and ROC-AUC would be misleadingly high."

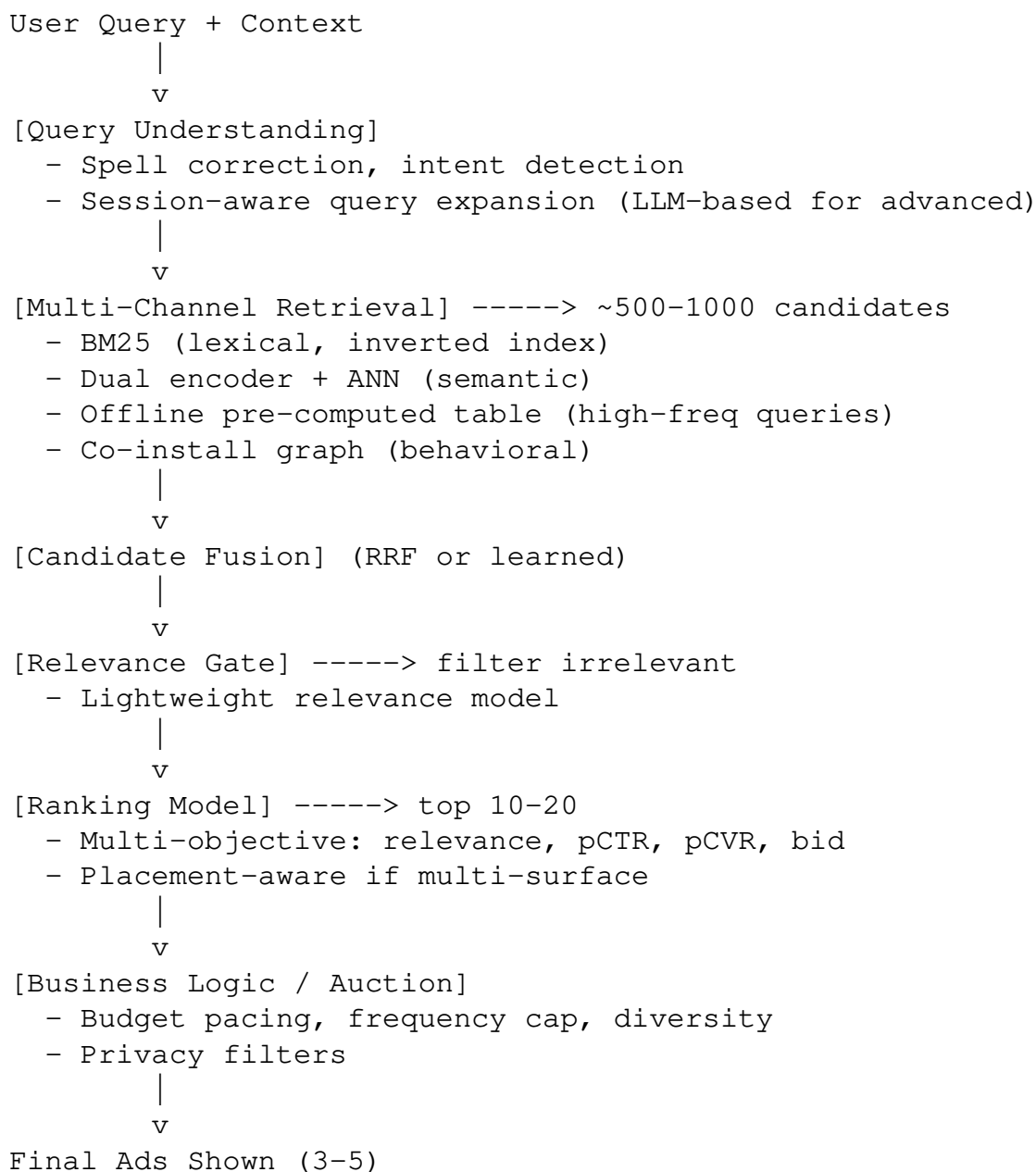
Online, the primary metrics are CTR and install rate for engagement, revenue per query for monetization, and relevance scores from human evaluation as a quality guardrail. We'd also monitor advertiser ROAS to ensure we're not sacrificing advertiser value for short-term clicks."

Step 3: High-Level Architecture (5 min)

How to say it:

"Let me draw the overall system architecture. It's a multi-stage funnel — each stage uses progressively heavier models on fewer candidates, trading off computation for quality."

Draw this:



How to say it:

"The key design principle: fewer candidates at each stage, heavier model per

candidate. Total end-to-end latency stays under 100ms. Let me now go deeper into the modeling choices for each stage, starting with retrieval since that's the core of sourcing. Would you like me to focus there, or is there a specific component you'd like to explore?"

This is where you let the interviewer steer. They'll likely say "Let's focus on the retrieval/matching model" or "Tell me about the ranking."

Why Multi-Channel? (Be ready to justify each channel)

If interviewer asks "Why not just use one retrieval method?":

How to say it:

"Each channel covers a different failure mode that the others miss. Let me explain why each one is needed:

BM25 (lexical): Handles exact keyword matching. When a user types 'Instagram,' we need exact match — a semantic model might retrieve other social media apps, but the user wants THIS specific app. BM25 also needs zero training data and zero ML infrastructure — it's our always-on baseline that works from day one.

Dual encoder + ANN (semantic): Handles the cases BM25 fundamentally cannot — synonym matching ('picture editor' finds 'Lightroom'), paraphrases ('app to edit my photos'), and conceptual matching ('make my selfie look better'). Any time there's no token overlap between query and app description, BM25 returns zero results. Semantic retrieval covers this entire gap.

Offline pre-computed table: For our top 10K queries that cover maybe 80% of traffic, we can afford to run the HEAVIEST possible model offline — even a cross-encoder scoring all 50K apps — and cache the results. This gives us maximum quality at zero serving latency for the head queries. We update the table daily.

Co-install graph: Captures behavioral patterns that neither lexical nor semantic similarity can detect. 'Users who installed Lightroom also installed Darkroom' — these two apps might have different descriptions and different keywords, but they serve the same user segment. This channel is especially valuable for discovery: surfacing apps the user didn't know to search for.

In practice, roughly 60% of good results come from semantic, 25% from BM25, 10% from offline table, and 5% from graph — but crucially, the marginal 5% from graph often contains the most surprising and valuable recommendations that no other channel would surface."

When each channel uniquely contributes (specific scenarios):

Scenario	Which channel saves us
"Instagram" (exact app name)	BM25 — navigational, exact match
"app to retouch portraits" (no token overlap with any app)	Semantic — understands intent
"photo editor" (top-10K query, well-studied)	Offline — uses heaviest model quality

User installed VSCO, what else? (no explicit query signal)
"camra editor" (misspelling)

Co-install graph — behavioral pattern
Semantic — subword tokenization handles OOV

Step 4: Data & Feature Engineering (5 min)

How to say it:

"Let me talk about what data we have and how we'd engineer features for training."

Training data sources:

Source	Signal type	Quality	Volume
App installs after search	Strong positive	High	Sparse
Ad clicks	Weak positive	Noisy (position bias)	Abundant
Impressions without click	Weak negative	Very noisy	Abundant
Human relevance judgments	Gold label	Highest	Expensive, limited
LLM-as-judge annotations	Strong	High	Scalable

Feature categories:

Category	Examples	Used in
Query features	Query text, length, intent type, embedding	Retrieval + Ranking
App/Ad features	Title, description, category, rating, developer, embedding	Retrieval + Ranking
Context features	Time of day, device type, OS version, session depth	Ranking
Behavioral (aggregated)	Category-level CTR, app popularity, historical install rate	Ranking
Cross features	Query-app similarity (BM25, cosine), category match	Ranking

Key data challenges to mention:

1. **Position bias** — clicks confounded by display position; use IPW
2. **Sparse conversions** — installs are rare; use multi-task learning with click as auxiliary
3. **Cold-start** — new apps have no behavioral data; rely on content features
4. **Privacy** — no persistent user profiles at Apple; use session-level + aggregated signals only

How to say it:

"For training the retrieval model, I'd use install logs as strong positives and clicks as weak positives, with inverse propensity weighting to debias the position effect. For negatives, a mix of random corpus samples, BM25 top

results not clicked as hard negatives, and in-batch negatives. The key data challenge at Apple is privacy — we can't build persistent user profiles, so features must be session-scoped or population-aggregated."

Step 5: Modeling Deep Dive (15 min)

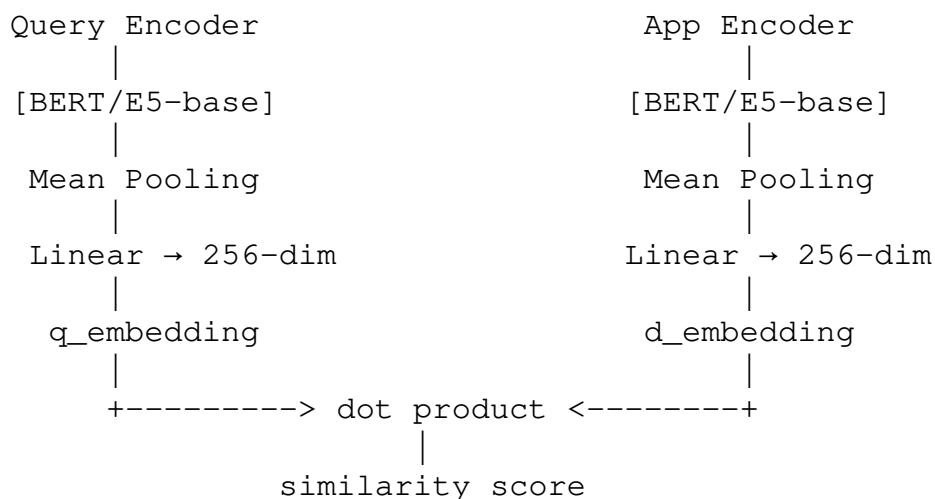
This is the core section. Cover retrieval model and ranking model.

5a: Retrieval — Dual Encoder

How to say it:

"For the retrieval model, I'd use a dual encoder architecture. Let me explain the architecture, training, and serving."

Architecture:



- Two independent encoders — query and app description encoded separately
- Output: dense embeddings in shared space
- Similarity: dot product (faster for ANN) or cosine

Architecture detail (if they ask "tell me more about the encoder"):

How to say it:

"For the encoder backbone, I'd start with a pre-trained text encoder like E5-base or sentence-transformers — these are already trained on large-scale text similarity tasks, so they give a strong initialization for our domain. The architecture is: input tokens go through a 12-layer transformer (BERT-style), we take the mean pool of the last hidden states to get a 768-dim representation, then a linear projection layer maps it down to 256 dimensions. The 256-dim choice balances expressiveness with ANN index size and search speed.

Both query encoder and app encoder share the same architecture but have SEPARATE weights — they're not weight-shared. This allows each encoder to specialize: the query encoder learns to handle short, ambiguous inputs; the app encoder learns to compress longer app descriptions into a single vector

that captures functionality.

One design choice: whether to use the same model for both sides or two different models. I'd start with the same architecture initialized from the same pre-trained weights, then let them diverge during fine-tuning. Some systems tie the weights (siamese) but I find asymmetric works better for query-document matching because the two inputs have fundamentally different distributions."

Training — Three approaches (simple → advanced):

Approach 1: Standard InfoNCE (baseline)

- Loss: InfoNCE (contrastive)

$$\mathcal{L} = -\log \frac{\exp(\mathbf{q}^T \mathbf{d} + \frac{1}{\tau})}{\exp(\mathbf{q}^T \mathbf{d} + \frac{1}{\tau}) + \sum_{i=1}^K \exp(\mathbf{q}^T \mathbf{d}_i - \frac{1}{\tau})}$$

- Temperature $\tau = 0.07$ controls how much model focuses on hard negatives
- All positives (clicks + installs) treated equally
- Simple but doesn't leverage label strength differences

Approach 2: Weighted/Multi-objective InfoNCE

$$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{click}} + \beta \cdot \mathcal{L}_{\text{install}} + \gamma \cdot \mathcal{L}_{\text{relevance}}$$

- Same InfoNCE formula, but computed on different positive sets with different weights
- Click pairs: weight 1.0 (noisy but abundant)
- Install pairs: weight 2.0 (strong signal, sparse)
- Relevance pairs (from human labels): weight 3.0 (highest quality, smallest set)
- All share the same embedding space — single dot product at serving time

How to say it:

"The multi-objective approach trains one embedding space but optimizes multiple loss terms simultaneously. The key constraint for retrieval is: at serving time we can only compute ONE dot product for ANN search. So unlike a ranking model that can have separate output heads, we must blend objectives at the loss level. I'd weight install-based loss higher than click-based because installs are a much cleaner signal of true relevance."

Approach 3: Curriculum / Multi-stage training (most sophisticated)

Stage 1: Pre-train on large click data (tens of millions of pairs)

- Noisy but gives broad coverage
- Model learns basic semantic matching
- Random negatives sufficient at this stage

Stage 2: Fine-tune on install data (millions of pairs)

- Stronger signal, sparser
- Add BM25 hard negatives
- Model refines toward conversion-relevant matching

- Stage 3: Fine-tune on human relevance labels (50K–200K pairs)
- Highest quality, smallest set
 - Add model-mined hard negatives (ANCE)
 - Model incorporates true relevance (not just behavioral proxy)

How to say it:

"I'd use a curriculum training approach — start with the noisiest but most abundant data to get a warm start, then progressively fine-tune on cleaner, sparser data. Stage 1 uses click logs with random negatives to learn basic semantic matching. Stage 2 fine-tunes on install data with harder negatives to learn conversion-relevant matching. Stage 3 does a final fine-tune on human relevance labels to align the embedding space with true relevance rather than just behavioral proxies.

Each stage uses a lower learning rate than the previous — we don't want to catastrophically forget what was learned in earlier stages. And at each stage, we increase the difficulty of negatives: random → BM25 hard negatives → model-mined hard negatives. This is both curriculum learning on data quality AND on negative difficulty."

Multi-objective vs Curriculum — Which is better? (Likely follow-up)

	Multi-objective (simultaneous)	Curriculum (staged)
How	One training loop, all data mixed, weighted loss	2-3 stages, each stage uses different quality data
Pro	Simple single pipeline, one run	Clean signal not drowned by noise, progressive refinement
Con	Noisy data (clicks) can overwhelm sparse clean data (relevance)	More complex, catastrophic forgetting risk
Best when	Data sources have similar volume/quality	Data quality gap is huge (10M clicks vs 50K labels)

Why multi-objective can fail with asymmetric data:

With 10M click pairs + 50K relevance labels trained simultaneously: even with $\gamma = 3\%$ on relevance, each epoch has 200x more gradient updates from clicks. The model is dominated by noisy click signal — like whispering in a loud bar.

Why curriculum works better here:

Stage 3 (50K relevance labels) fine-tunes an already well-initialized model. The 50K is enough because the model already learned good representations from Stages 1-2. And the clean signal isn't competing with 10M noisy gradients — they're in separate training phases.

Optimal: Curriculum across stages + multi-objective within stages:

Stage 1: Click data only (warm start)
 $\text{Loss} = \text{InfoNCE}(\text{click_positives})$

Stage 2: Install + Click (multi-objective within this stage)
 $\text{Loss} = \alpha \cdot \text{InfoNCE}(\text{install_pos}) + \beta \cdot \text{InfoNCE}(\text{click_pos})$

Data quality more balanced now; both signals useful

Stage 3: Relevance + Install (multi-objective within this stage)
Loss = $\alpha \cdot \text{InfoNCE}(\text{relevance_pos}) + \beta \cdot \text{InfoNCE}(\text{install_pos})$
Cleanest data dominates; install provides regularization

How to say it in interview:

"I'd use curriculum training over pure multi-objective for retrieval. The reason is data quality asymmetry — we have tens of millions of noisy click pairs but only 50K clean relevance labels. Training them simultaneously means click gradients overwhelm relevance regardless of how we weight the loss — it's 200x more updates per epoch.

Curriculum avoids this: warm start on abundant noisy data, then progressively fine-tune on cleaner data with lower learning rates. Each stage builds on the previous without being contaminated by it.

Within each stage though, I might still use multi-objective — for example, Stage 2 combines install loss with click loss at reduced weight for regularization. So curriculum controls the data schedule across stages, and multi-objective controls optimization within a stage. They're complementary, not competing approaches."

Additional training details:

- Negative sampling: in-batch negatives + BM25 hard negatives + model-mined (ANCE)
- Position bias: IPW on click-based positives (clicks from lower positions get higher weight)

Why dual encoder (not cross-encoder):

- Must search 50K+ candidates in <10ms
- Pre-compute app embeddings offline → ANN search online
- Cross-encoder for ranking later (on 100-500 candidates)

Embedding serving & cosine guarantee:

How to say it (this came up in ML round):

"To ensure embeddings are valid for cosine similarity, both encoders must be co-trained — they share the same contrastive loss so their outputs live in the same embedding space. At serving time, I'd L2-normalize the embeddings before storing them in the ANN index. This guarantees that dot product equals cosine similarity, and all scores fall in [-1, 1]. The normalization happens once at index-build time for app embeddings, and once per query at inference time."

ANN serving:

Offline:

1. Encode all apps → 256-dim embeddings
2. L2-normalize

3. Build HNSW index (M=32, ef_construction=200)

Online:

1. Encode query → 256-dim embedding, L2-normalize
2. HNSW search → top-500 candidate IDs (~3-5ms)
3. Pass to fusion/ranking

5b: Ranking — Multi-Objective Model

How to say it:

"For ranking, I'd use a deeper model that scores each candidate independently using rich features — not just embeddings but also behavioral, contextual, and business signals."

Architecture options:

Option	When to use
Feature-rich DNN (DeepFM, DCN-v2)	Standard case, lots of features
Multi-task with shared bottom (MMOE/PLE)	Multiple objectives (CTR + CVR + relevance)
PEPNet-style gating	Multiple placements/surfaces with different behavior

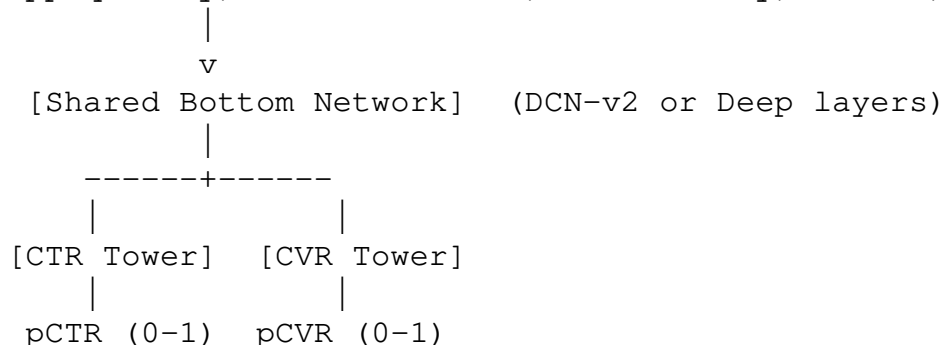
Key clarification: Multi-head MODEL vs Multi-objective SCORING

These are two different things that work together:

The MODEL is multi-head (what we TRAIN):

Input Features:

query_emb, app_emb, cosine_sim, category_match,
app_quality, historical_CTR, time_of_day, device, ...



- **CTR tower:** trained with label = did user click? (binary cross-entropy)
- **CVR tower:** trained with label = did user install? (binary cross-entropy, only on clicked samples)
- **Shared bottom:** learns common representations useful for both tasks
- Training loss: $\mathcal{L} = \lambda_1 \cdot \text{BCE}(\text{pCTR}, y_{\text{click}}) + \lambda_2 \cdot \text{BCE}(\text{pCVR}, y_{\text{install}})$

This is a true multi-head model — two prediction heads, two different labels, one shared backbone.

The SCORING FORMULA is how we COMBINE predictions at serving time (not trained):

$$\text{score} = w_1 \cdot \text{pCTR} + w_2 \cdot \text{pCVR} + w_3 \cdot \text{relevance} + w_4 \cdot \text{bid} - w_5 \cdot \text{diversity penalty}$$

- pCTR : output from CTR tower (model prediction)
- pCVR : output from CVR tower (model prediction)
- relevance : output from separate relevance model (not part of ranking model)
- bid : advertiser's bid price (business input, not a model prediction)
- diversity penalty : computed from already-shown ads (real-time business logic)
- $w_1 \dots w_5$: tunable weights, set per-placement, NOT learned by gradient descent

How to say it:

"Let me clarify the architecture. The ranking MODEL itself is multi-head: it has a shared bottom network processing all features, with separate towers for pCTR and pCVR — each trained with its own binary label. The CTR tower learns 'will this user click this ad' from click logs. The CVR tower learns 'will this user install this app' from install logs.

Now, the final RANKING SCORE is a separate step — it's not a learned function, it's a business-logic combination of the model's predictions plus external signals. We take pCTR and pCVR from our model, relevance score from a separate relevance model, the advertiser's bid price, and a diversity penalty — and combine them with hand-tuned weights. The bid and relevance are input signals, not things the ranking model predicts.

Why separate the model from the scoring formula? Because the weights need to be tunable at the business level without retraining the model. If we want to increase relevance emphasis for a new surface, we just change w_3 — we don't retrain the model. If a new advertiser category bids differently, the model doesn't need updating."

Why not put bid/relevance AS INPUT FEATURES to the model instead?

"You could — and some systems do use bid as a feature. But there's a risk: the model might learn to over-index on bid price, essentially learning 'higher bid = always rank higher' which defeats the purpose of relevance and quality scoring. By keeping the scoring formula explicit and auditable outside the model, we maintain direct control over the trade-offs. Also, bid prices change in real-time (auction dynamics), and we don't want to retrain the model every time bidding patterns shift."

Training the ranker:

- Labels: click (binary, for pCTR tower), install (binary, for pCVR tower)
- Loss: $\mathcal{L} = \lambda_1 \cdot \text{BCE}(\hat{y}_{\text{ctr}}, y_{\text{click}}) + \lambda_2 \cdot \text{BCE}(\hat{y}_{\text{cvr}}, y_{\text{install}})$
- Position bias: position as feature at training, set to constant at inference
- Negative sampling: all impressed-but-not-clicked in top positions

- CVR tower special: only trained on clicked samples (can't have install without click)

Weights $w_1 \dots w_5$ — how are they tuned?

- NOT gradient descent. These are business-level parameters.
 - Method 1: Grid search on offline counterfactual replay (simulate different weight combinations, evaluate NDCG + revenue)
 - Method 2: Online A/B test different weight configurations
 - Method 3: Bayesian optimization over the weight space
 - Typical: start with heuristic (e.g., $w_1 = 0.3$, $w_2 = 0.3$, $w_3 = 0.2$, $w_4 = 0.15$, $w_5 = 0.05$), then tune online
-

5c: Relevance Model (Quality Gate)

How to say it:

"Between retrieval and ranking, I'd add a relevance gate — a dedicated model that filters out irrelevant candidates regardless of their revenue potential."

- Architecture: distilled cross-encoder (trained on human labels, distilled to lightweight student)
 - Labels: human annotation (50K pairs) → cross-encoder teacher → 5M soft labels → student MLP/GBDT
 - Serving: student model, ~0.1ms per candidate
 - Threshold: relevance < 0.3 → filter out
-

Step 6: Serving & Infrastructure (7 min)

How to say it:

"Let me talk about how this system actually serves in production under latency and reliability constraints."

Latency budget breakdown:

Component	Budget	How
Query understanding	5ms	Lightweight classifier + lookup
Query encoding (dual encoder)	10ms	GPU inference on query encoder
ANN search	5ms	HNSW in-memory
Relevance scoring (500 cands)	10ms	Batch inference, student model
Ranking (200 cands)	20ms	Feature lookup + DNN inference
Business logic	5ms	Rule engine
Total	~55ms	Under 100ms budget

Key infrastructure decisions:

1. Embedding index updates:

- New apps: compute embedding, add to index (incremental, no full rebuild)
- Model update: full re-index offline, hot-swap the index
- Cadence: daily model updates, real-time new-app ingestion

2. Feature store:

- Offline features: precomputed daily (app quality scores, historical CTR)
- Near-real-time features: session-level signals (last 5 queries, click patterns)
- Online features: current query, time of day, device

3. Fallback & reliability:

- If ANN index fails → serve from BM25 only
- If ranking model fails → serve by retrieval score
- Circuit breaker: if latency exceeds budget, short-circuit to cached results

4. Model update safety (from ML round — they care about this!):

- Shadow scoring: new model scores in parallel, compare output distributions before promoting
- Canary deployment: serve new model to 1% traffic, monitor for anomalies
- Automatic rollback: if null rate > 0.1% or score distribution KL-divergence > threshold

How to say it:

"For reliability, I'd implement shadow scoring before any model promotion — the new model scores alongside the current one, and we compare output distributions. If KL-divergence exceeds a threshold or null rates spike, the promotion is blocked. This catches exactly the type of issue you mentioned earlier — where incremental updates cause gradient explosion or layer outputs going null. The key is catching it BEFORE it hits production traffic."

6b: Production Monitoring — Feature Drift & Model Health

This is a critical topic for teams with daily incremental model updates (which this team does).

What can drift and why it matters:

What drifts	Consequence	Example
Input feature distribution	Model sees OOD data → garbage predictions	A new app category floods the system
Label distribution	Model's calibration breaks	Seasonal shift (holiday → install rate spikes)
Embedding space	ANN retrieval quality degrades	Model update shifts embedding distribution
Prediction output distribution	Business metrics degrade	Score inflation → all candidates look equally good

Monitoring Numerical Features — Statistical Tests

For each input feature, monitor the distribution shift between training data and live serving data:

Method 1: Population Stability Index (PSI)

$$\text{PSI} = \sum_{i=1}^B (p_i - q_i) \cdot \ln \frac{p_i}{q_i}$$

- p_i = proportion of values in bin i for the current (serving) distribution
- q_i = proportion of values in bin i for the reference (training) distribution
- Split feature values into $B = 10$ equal-width bins
- $\text{PSI} < 0.1$: no significant drift
- $\text{PSI} 0.1-0.25$: moderate drift, investigate
- $\text{PSI} > 0.25$: significant drift, alert

Method 2: Kolmogorov-Smirnov (KS) Test

$$D = \max_x |F_{\text{train}}(x) - F_{\text{serve}}(x)|$$

- Compares the CDFs of training vs serving distributions
- $D > \text{critical value}$ (depends on sample size) $\rightarrow$ significant drift
- Pro: non-parametric, no binning needed
- Con: sensitive to sample size (always rejects with enough data)

Method 3: Wasserstein Distance (Earth Mover's Distance)

$$W(P, Q) = \inf_{\gamma \in \Gamma(P, Q)} \mathbb{E}_{(x, y) \sim \gamma}[|x - y|]$$

- Intuitively: minimum "work" to transform one distribution into another
- More interpretable than KS for continuous features (in original units)
- Good for tracking gradual drift over time (plot daily)

Practical setup:

For each numerical feature (e.g., `app_rating`, `historical_ctr`, `query_len`)

1. Compute reference distribution from training data
2. Every hour: compute PSI between last hour's serving data and reference
3. Alert if $\text{PSI} > 0.25$ for any critical feature
4. Dashboard: plot daily Wasserstein distance trend per feature

Monitoring Categorical Features

For categorical features (e.g., `app_category`, `device_type`, `query_intent_type`):

- Track frequency distribution: $P(\text{category} = c)$ for each category
- Alert on:
 - New unseen categories appearing (OOV)
 - Existing category proportion shifting $> 20\%$ relative
 - Top-K categories changing rank order

- Method: chi-squared test or simple proportion monitoring
-

Monitoring Embeddings — Drift in Vector Space

Embeddings are harder to monitor because they're high-dimensional. Standard statistical tests don't work well on 256-dim vectors.

Method 1: Centroid Drift

Reference: compute centroid of all app embeddings from last stable mode
`centroid_ref = mean(all_app_embeddings)`

Daily: compute centroid of new embeddings after model update
`centroid_new = mean(all_app_embeddings_new)`

`Drift = cosine_distance(centroid_ref, centroid_new)`
Alert if drift > threshold (e.g., 0.1)

- Simple, cheap to compute
- Catches global distribution shift
- Misses local changes (subset of embeddings drifted)

Method 2: Nearest Neighbor Consistency

For a sample of 1000 apps:

1. Retrieve top-10 nearest neighbors using OLD embeddings
2. Retrieve top-10 nearest neighbors using NEW embeddings
3. Compute Jaccard overlap: $|\text{old_NN} \cap \text{new_NN}| / |\text{old_NN} \cup \text{new_NN}|$

Average Jaccard < 0.7 → significant embedding drift, investigate

- Directly measures "did the model's notion of similarity change?"
- More robust than centroid (catches local changes)
- Moderate compute cost (sample-based)

Method 3: Cosine Similarity Distribution Shift

For a sample of 10K (query, app) pairs:

1. Compute cosine similarity using OLD model
2. Compute cosine similarity using NEW model
3. Plot distribution of `sim_old` vs `sim_new`
4. Compute KS-test or PSI between the two distributions

If score distribution shifts significantly → model behavior changed

- Directly monitors what the ANN search will return
- Catches calibration issues (e.g., new model scores everything higher)

Method 4: Dimensionality Monitoring (PCA/UMAP)

1. Project embeddings to 2D via PCA (use same PCA fitted on reference)
2. Compare cluster structure: are the same clusters present?

3. Check if variance explained by top-K PCs changed
(if it dropped → embeddings became less structured)
- Visual monitoring on dashboard
 - Catches embedding space collapse (all vectors converging)
-

Monitoring Model Output Distribution

Track prediction score distributions hourly:

What to monitor	How	Alert condition
Mean prediction score	Running average	Shift > 2 standard deviations
Score variance	Running variance	Collapse (variance → 0) or explosion
Null/NaN rate	Count nulls / total	> 0.01%
Score percentiles (p10, p50, p90)	Quantile tracking	p90-p10 spread changes > 30%
KL-divergence vs yesterday	Daily comparison	KL > 0.05

Complete Monitoring Stack (what to say in interview)

How to say it:

"For production monitoring, I'd set up three layers of drift detection.

First, input feature monitoring: for each numerical feature, I'd compute Population Stability Index hourly against the training distribution. PSI above 0.25 triggers an alert. For categorical features, I'd track proportion shifts and flag new unseen categories. This catches data pipeline issues — if a feature starts arriving as null, or a new app category floods the system.

Second, embedding space monitoring: since our retrieval relies on embedding similarity, I need to know if the embedding space shifted after a model update. I'd use nearest-neighbor consistency — for a sample of apps, check if their top-10 neighbors changed significantly between the old and new model. If Jaccard overlap drops below 0.7, that's a red flag that the model's notion of similarity changed in unexpected ways.

Third, output distribution monitoring: track the prediction score distribution hourly — mean, variance, percentiles, null rate. If KL-divergence between today's and yesterday's score distribution exceeds 0.05, or if null rate spikes above 0.01%, that triggers automatic model rollback. This is exactly what catches the kind of incremental update failure we discussed earlier — gradient explosion causing null outputs would show up immediately as a null rate spike.

The key principle: catch drift at the INPUT level before it becomes a problem at the OUTPUT level. If features are drifting, the model hasn't failed yet but it will soon."

Step 7: Evaluation & Iteration (5 min)

How to say it:

"Finally, let me talk about how we'd evaluate this end-to-end and iterate."

Offline evaluation:

What to evaluate	Method
Retrieval recall	Hold-out: can model retrieve the apps user eventually installed?
Ranking quality	NDCG on human-labeled relevance judgments
Relevance model	PR-AUC on annotation test set
End-to-end	Simulated serving with replay of historical queries

Online evaluation (A/B test design):

- Traffic split: 5% initial → 15% → 50% ramp
- Guardrail metrics: relevance score, null rate, latency p99
- Primary metrics: CTR, install rate, revenue per query
- Duration: 2 weeks minimum for significance
- Novelty effect: check if gains persist after week 1

Iteration priorities:

1. First: get basic BM25 + simple dual encoder working end-to-end
2. Then: add hard negatives, improve dual encoder quality
3. Then: add relevance gate + ranking model
4. Then: add multi-channel retrieval (offline table, graph)
5. Then: LLM-based query understanding for session personalization
6. Then: placement-aware ranking for multi-surface

How to say it:

"I'd take an iterative approach — start with the simplest viable pipeline, measure baselines, then add components incrementally. Each addition should show measurable improvement in both offline metrics and online A/B tests. The first priority is retrieval recall — if we can't find the relevant apps, nothing downstream can save it."

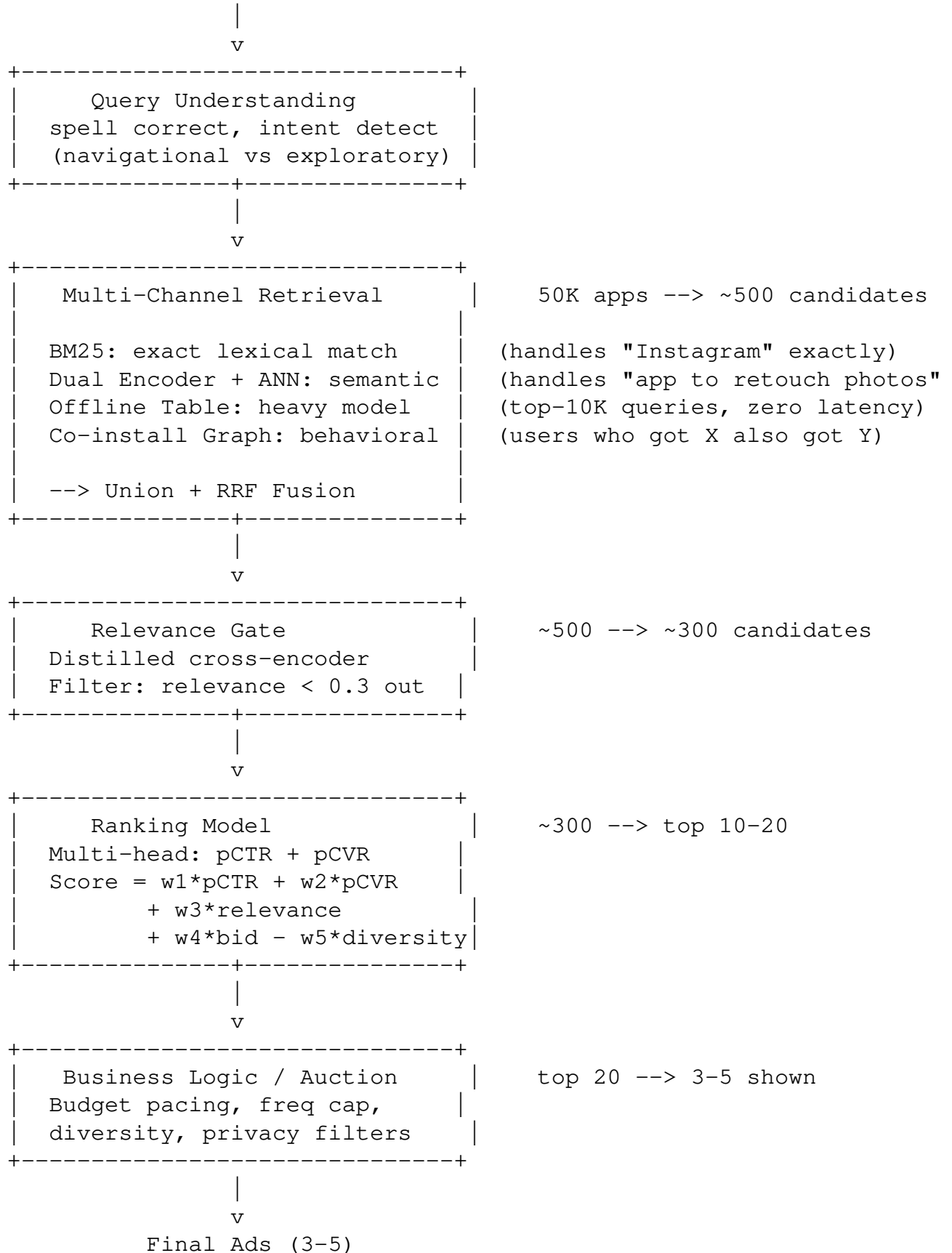
Likely Problem Prompts for Apple Ads

Based on recruiter context ("recommendation systems, personalization, search, retrieval ranking, ads matching"):

Possible prompt	Key focus areas
"Design an ad matching system for App Store search"	Retrieval + ranking, privacy constraints
"Design a recommendation system for App Store browse page"	Personalization without user profiles, cold-start

Draw this on the whiteboard as you talk:

User Query: "photo editor" + Session Context



"The overall system is a multi-stage funnel. Let me walk through each layer.

Query understanding handles spell correction and intent detection. This is

critical because navigational queries like 'Instagram' should shortcut to exact match — the user wants THAT specific app. Exploratory queries like 'photo editor' need the full retrieval pipeline.

Multi-channel retrieval runs four parallel paths. Why multiple channels? Because each covers a different failure mode. BM25 handles exact keyword matches — when query tokens appear literally in app names, nothing beats it. But BM25 returns zero results if there's no token overlap — that's where the dual encoder with ANN comes in, matching on semantic meaning. For our top 10K queries covering 80% of traffic, we can afford to pre-compute results using our heaviest model offline and cache them — zero serving latency, maximum quality. And the co-install graph captures behavioral patterns invisible to text matching: 'users who installed Lightroom also installed Darkroom' — no text similarity, but same user segment.

We merge all channels using Reciprocal Rank Fusion — it only uses ranks, not raw scores, so we don't need to calibrate across channels. This gives about 500 candidates.

Relevance gate: a lightweight distilled model filters out anything below a relevance threshold — this is our quality guardrail that can't be overridden by revenue.

Ranking model: a multi-head network predicting pCTR and pCVR, combined with relevance score, bid price, and diversity penalty via a tunable scoring formula. The model predicts engagement; the formula blends it with business objectives.

Business logic applies budget pacing, frequency caps, and privacy filters.

Total end-to-end latency: under 100ms. Each stage uses progressively heavier models on fewer candidates."

[Pause] "Should I dive into data and training, or go straight to the retrieval model architecture?"

Data & Features (3 minutes, if they ask or you offer):

"Let me cover what training data and features we'd use.

Training data for the retrieval model comes from search logs. I have three quality tiers of positives: installs are strong positives — the user actively chose to download. Clicks are weak positives — noisy because of position bias and curiosity clicks. And if budget allows, human relevance annotations or LLM-as-judge labels give us ground truth relevance that's independent of behavioral confounds.

For negatives: random corpus samples give easy baseline contrast but the model learns them instantly. BM25 top results that weren't clicked — these are medium-hard negatives because they match lexically but the user rejected them. In-batch negatives come for free. And for the hardest negatives, I'd mine from the model's own top predictions that are actually irrelevant — the ANCE

approach.

A critical data issue is **position bias**: apps in position 1 get clicked more regardless of quality. I'd use inverse propensity weighting — compute average CTR per position from logs, then weight each click sample by 1 over that position's CTR. A click from position 8 gets much higher training weight because the user overcame a position disadvantage.

Training strategy: I'd use curriculum training rather than mixing all data together. Stage 1: pre-train on abundant click data with random negatives — learn basic semantic matching. Stage 2: fine-tune on install data with BM25 hard negatives — refine toward conversion-relevant matching. Stage 3: fine-tune on human relevance labels with model-mined hard negatives — align with true relevance. Each stage uses lower learning rate and harder negatives. This avoids the problem of noisy click data overwhelming sparse clean labels — which would happen if we trained multi-objective simultaneously.

Features for the ranking model span multiple categories. Query features: the query embedding, length, detected intent type. App features: app embedding, category, rating, developer reputation, description quality score. Context features: time of day, device type, session depth. Behavioral features at population level: category-level CTR, app-level historical install rate. And cross features: query-app cosine similarity from the retrieval model, BM25 score, category match — these are signals from the upstream stages that become features downstream.

Given Apple's privacy constraints, all behavioral features must be aggregated at population level — no individual user profiles. Session-level features like 'which apps has THIS user browsed in THIS session' are fine since they're first-party and ephemeral."

Modeling deep dive — Retrieval (5-7 minutes, when they ask):

"For the retrieval model, I'd use a dual encoder architecture. Let me walk through architecture, training, and serving.

Architecture: Two separate encoders — a query encoder and an app encoder. Each is a 12-layer transformer initialized from a pre-trained text model like E5-base. The transformer outputs are mean-pooled to get a 768-dim representation, then a linear projection maps down to 256 dimensions. The two encoders have the same architecture but separate weights — they're not weight-tied. This lets each specialize: the query encoder handles short, ambiguous inputs; the app encoder compresses longer descriptions into a single vector capturing functionality.

The 256-dim choice balances expressiveness with practical constraints — at 50K apps, the full index is 50K times 256 times 4 bytes, about 50MB, fitting comfortably in memory.

Why dual encoder over cross-encoder: servability. We pre-compute all 50K app embeddings offline and store them in an HNSW index. At query time, we only need one forward pass through the query encoder — about 10ms on GPU

— then an ANN lookup in 3-5ms. A cross-encoder would require 50K forward passes, one per candidate — physically impossible in 100ms. We use cross-encoder later in the relevance gate, where candidates are already reduced to a few hundred.

Training: The loss is InfoNCE — a softmax over similarities. For each query, we have one positive app and K negatives. The model maximizes the probability of selecting the correct positive from the set. Temperature tau at 0.07 controls how much the model focuses on hard negatives versus easy ones — lower temperature means sharper focus on the hardest cases.

I'd train with curriculum: first stage on click data with random negatives to warm start, second stage on install data with BM25 hard negatives, third stage on relevance labels with model-mined negatives. Each stage progressively refines the embedding space with cleaner signal and harder contrast.

Embedding serving guarantee: I'd L2-normalize all embeddings before indexing. This ensures dot product equals cosine similarity, scores are bounded in [-1, 1], and magnitude doesn't dominate direction. Both encoders are co-trained with the same contrastive loss, so their outputs live in the same embedding space — this is what makes the cosine similarity between a query and an app embedding meaningful. You can't take embeddings from two different models and expect cosine to work.

ANN index: HNSW with M = 32 neighbors per node and ef_search = 64. This gives about 99% recall versus exact brute-force search. The index is rebuilt daily after model updates via hot-swap — the new index is constructed offline and atomically replaces the old one. New apps get their embedding computed and added incrementally without full rebuild."

Modeling deep dive — Ranking (3-5 minutes, if they ask):

"For ranking, I need to score each of the ~300 candidates that passed the relevance gate. The model is a multi-head architecture with a shared bottom.

Architecture: The shared bottom is a DCN-v2 style network processing all features — query embedding, app embedding, their cosine similarity, category match, app quality scores, historical engagement rates, context features. On top of the shared layers, I have two task towers: one for pCTR and one for pCVR. Each is trained with its own binary cross-entropy loss — CTR tower on click labels, CVR tower on install labels (only for clicked samples, since you can't install without clicking first).

Important clarification: the multi-head model predicts pCTR and pCVR. The final ranking SCORE is a separate serving-time formula that COMBINES these predictions with external signals:

score = w1 times pCTR plus w2 times pCVR plus w3 times relevance plus w4 times bid minus w5 times diversity penalty.

Here, pCTR and pCVR come from our model. Relevance comes from the upstream relevance gate — it's an input, not something the ranker predicts.

Bid is the advertiser's price — a business signal. Diversity penalty is computed from what's already been shown.

Why separate the model from the formula? Because the weights are business-level parameters that product managers tune without retraining the model. If we want to emphasize relevance more on a new surface, we change w_3 . If bidding dynamics shift, the model doesn't need updating. The weights are tuned via offline counterfactual replay and online A/B tests.

For multi-surface: If we expand to Apple News, Maps, etc., I'd add PEPNet-style gating — a learned placement embedding that element-wise gates the shared representation. This lets the model share statistical strength across surfaces (critical for cold-start placements with little data) while adapting its behavior per-surface through the gate."

Serving, Monitoring & Reliability (2-3 minutes):

"For production, three critical layers.

Model update safety: We run daily incremental updates. Before promotion, I'd implement shadow scoring — the new model scores in parallel with production. We compare output distributions: KL-divergence, null rate, score percentile shifts. If KL exceeds 0.05 or null rate exceeds 0.01%, the update is blocked and rolls back automatically. This catches gradient explosion issues before they hit users.

Feature drift monitoring: For each numerical input feature, I'd compute Population Stability Index hourly — comparing serving distribution against training distribution. PSI above 0.25 triggers investigation. For embeddings specifically, I'd monitor nearest-neighbor consistency: for a sample of apps, check if their top-10 neighbors changed after a model update. Jaccard overlap below 0.7 means the embedding space shifted significantly.

Fallback chains: If semantic retrieval fails, BM25 still serves. If ranking fails, we serve by retrieval score. If the whole system is overloaded, we short-circuit to cached results for popular queries. Users never see an error — degraded quality is always better than no results.

The latency budget: query encoding 10ms, ANN search 5ms, relevance scoring 10ms, ranking 20ms, business logic 5ms — total about 55ms well within the 100ms budget."

Challenge Questions & How to Respond

Based on ML round patterns — this interviewer digs into operational details, failure modes, and embedding mechanics.

"What if the model starts outputting nulls after an incremental update?"

"This is a training stability issue in the incremental pipeline. The most likely

causes are:

First, data corruption in the incremental batch — a bad data partition with missing features or corrupted labels that causes gradient explosion in specific layers.

Second, learning rate schedule mismatch — if the incremental update uses the same learning rate as initial training, it can overshoot when fine-tuning on a small daily batch.

Third, batch normalization statistics drift — if the daily batch has a very different distribution, BN running statistics can shift dramatically, causing downstream activations to explode or collapse.

For debugging: I'd check gradient norms per layer across the update, compare activation statistics (mean, variance) of each layer before and after the update, and validate the training data batch for anomalies (missing values, extreme outliers). For prevention: gradient clipping, validation on a hold-out set before promoting the model, and shadow scoring in production."

"How do you ensure embeddings work for cosine similarity?"

"Three guarantees are needed. First, co-training: both query and app encoders are trained with the same contrastive loss, so their outputs live in the same embedding space. You can't take embeddings from two different models and compute meaningful cosine between them.

Second, L2-normalization: I normalize all embeddings to unit length before storing. This means dot product equals cosine similarity, and scores are bounded in $[-1, 1]$. It also prevents magnitude from dominating similarity — two vectors with different norms but same direction should be equally similar.

Third, the embedding space must be calibrated so that cosine $>$ some threshold actually means 'relevant.' We validate this by checking: are the top-K by cosine similarity actually relevant apps? If the calibration is off, we add a learned scaling layer or tune the temperature parameter in the contrastive loss."

"What if retrieval recall is good but end-to-end CTR doesn't improve?"

"This means the retrieval model is finding relevant candidates, but the ranking model isn't surfacing them correctly. Possible causes:

First, feature leakage in the ranker — it might be relying heavily on historical CTR features, which favor popular apps over newly relevant ones from improved retrieval.

Second, the ranking objective is misaligned — if it optimizes purely for pCTR, it might prefer clickbait over truly relevant ads. Need to add relevance as an explicit objective.

Third, position bias in ranking training data — the ranker learns to replicate

the old system's ordering because training labels come from the old system's impressions.

I'd debug by: comparing the ranking of new retrieval candidates vs existing ones, checking if the ranker's relevance score correlates with the retrieval model's similarity score, and running a counterfactual analysis — if we force-served the new retrieval candidates in top positions, would they get clicked?"

"How do you handle cold-start for new apps with no engagement data?"

"Multiple layers of defense:

The dual encoder handles this naturally — it's trained on text, so a new app's title and description produce a meaningful embedding even with zero click history.

BM25 works immediately once the app's metadata is indexed.

For ranking, I'd use content-based features as priors: developer reputation, category average CTR, app description quality score, visual quality score. These are available for new apps from day one.

For exploration, I'd allocate a small impression budget using Thompson sampling — give new apps a chance to collect engagement signal while limiting the revenue risk."

"Why not just use one big end-to-end model instead of multi-stage?"

"Three reasons. First, latency: scoring 50K apps with a heavy model in < 100ms is physically impossible. Multi-stage lets us use a 1ms-per-candidate model on 50K, then a 5ms model on 500.

Second, debuggability: if CTR drops, I can isolate whether it's a retrieval problem (recall dropped) or a ranking problem (recall is fine but ordering is wrong).

Third, different optimization objectives at different stages: retrieval optimizes for recall (don't miss relevant ads), ranking optimizes for precision and business objectives (show the best ones). Forcing one model to do both leads to neither being done well."

"How would you incorporate LLMs into this system?"

"LLMs add value at three points:

First, query understanding: using an LLM to infer user intent from session behavior and generate refined queries that guide retrieval. For example, if a user searched 'photo editor' and browsed Lightroom and Photoshop, the LLM infers 'professional color grading tool' and generates specific child queries for semantic retrieval.

Second, relevance labeling: using LLM-as-judge to generate relevance

annotations at scale — replacing expensive human annotation for training the relevance model.

Third, app understanding: using LLMs to extract structured attributes from app descriptions — functionality, target audience, use cases — as features for the ranking model.

The key constraint is latency: LLM inference must either be async/pre-computed, or use a small distilled model (<7B) with strict timeout and fallback. For the query understanding use case, I'd add an intent detection gate — only invoke the LLM when there's enough session signal to make personalization worthwhile."

"How do you balance revenue and relevance?"

"This is the fundamental tension in ads. My approach:

Hard constraint: a relevance minimum threshold that cannot be overridden by revenue. If an ad isn't relevant, it doesn't show regardless of bid.

Soft optimization: within the 'relevant' candidate set, the ranking score blends relevance with monetization objectives using tunable weights.

Monitoring: I'd track a relevance-revenue trade-off curve and alert if the operating point shifts — for example, if a model update improves revenue but drops relevance scores.

For Apple specifically, the premium brand positioning means relevance should have higher weight than at a pure performance ads platform. The user experience is a moat."

Signals from ML Round

Based on today's ML knowledge round, here's what this team cares about (incorporate into your system design):

Signal from ML round	What it tells you	How to use in system design
Model debugging / null outputs	They run daily incremental model updates and have hit stability issues	Emphasize model update safety: shadow scoring, gradient monitoring, automatic rollback
Embedding cosine guarantee	They use embedding-based retrieval in production	Show you understand: L2-norm, co-training, calibration
"Intersection" of embeddings	They think about how to serve/index embeddings	Cover ANN index mechanics, incremental updates, hot-swap
LLM sourcing mention	They're actively working on LLM-based retrieval	Show your LLM orchestration experience as directly relevant
Layer-by-layer debugging	They expect practical debugging skills	Mention gradient norm checks, activation statistics, validation gates

Key takeaway: This team values **production ML engineering** as much as modeling. They want to see you can not only design a model but also deploy, monitor, and debug it in production with daily updates. Make sure your system design includes the operational layer, not just the algorithm.
